整体过程

我的项目主要是基于 STM32G431 实现永磁同步电机的有感 FOC 矢量控制，采用位置环、速度环和电流环三级串级控制结构，其中电流环运行频率为 16kHz，速度环为 1kHz，位置环为 200Hz。整个控制过程首先由上位机或者 ESP32-S3 人机交互界面通过 CAN 总线发送目标转速或目标位置。在速度控制模式下，速度环将目标转速与编码器反馈的实际转速进行比较，通过 PI 控制器计算得到 q 轴目标电流 i_q^*，同时将 d 轴目标电流 i_d^* 设置为零，实现最大转矩电流比控制在表贴式永磁同步电机上的常用简化方式。接下来进入电流环，利用 STM32G431 的 TIM1 产生三相互补 PWM，并通过 TIM1 的 CH4 通道同步触发 ADC1 和 ADC2 的注入转换，采集电机 A、B 两相电流，根据三相电流之和为零的关系重构 C 相电流。随后通过 Clarke 变换将三相电流转换到两相静止坐标系，再结合编码器反馈的转子电角度进行 Park 变换，得到 d、q 轴实际电流。将实际电流与目标电流进行比较，分别通过两个 PI 控制器计算得到 d、q 轴目标电压 u_d 和 u_q，经过母线电压限幅后，再通过反 Park 变换转换为静止坐标系下的 u_α 和 u_β。之后采用 Min-Max 法实现 SVPWM，通过注入公共零序分量计算三相桥臂的 PWM 占空比，并将结果写入 TIM1 的 CCR1、CCR2 和 CCR3 比较寄存器，最终驱动三相逆变桥产生相应的电压，控制电机转矩和转速。与此同时，编码器通过 TIM3 的正交编码器模式获取转子位置，并通过计数差分和低通滤波计算实际转速，形成完整的闭环反馈。如果采用位置控制模式，则在速度环外增加位置环，通过位置误差计算目标转速，再依次经过速度环和电流环实现位置跟踪。此外，项目还设计了电机启动状态机，在进入闭环控制前完成编码器电角度零点校准，并利用 CAN 通信和 VOFA+ 实现电机启停、正反转控制、参数调节以及运行数据监测。

电流过程

在我的 ZqFOC 项目中，电流采样功能主要由 foc_current.c 文件实现，其核心作用是采集电机的相电流，将 ADC 数字量转换为实际电流值，并为后续 Clarke、Park 变换及电流闭环控制提供反馈。这个文件主要包含三个函数。第一个是 FOC_CURRENT_Init()，负责初始化电流采样结构体，关联 ADC1 和 ADC2 的外设句柄，设置电流采样的初始零点偏置，并将电流变量初始化为零。第二个是 FOC_CURRENT_CalibrateOffset()，主要负责上电时的零电流偏置校准。由于实际电路中的运放存在偏置电压，即使电机没有电流，ADC 采样结果也不一定为零，所以需要在三相功率输出关闭的情况下，利用 TIM1 的 CH4 通道触发 ADC 注入转换，连续采集 24 组 A、B 相电流采样值，分别计算平均值，作为两相电流的零点偏置，以减小硬件偏置带来的测量误差。第三个是 FOC_CURRENT_Update()，这是电机正常运行时执行的电流更新函数。在硬件配置上，我使用 STM32G431 的 TIM1 产生 16kHz 的中心对齐 PWM，同时利用 TIM1_CH4 作为 ADC 注入组的外部触发源，使 ADC1 和 ADC2 在指定的 PWM 时刻进行采样，其中 ADC1 采集 A 相电流，ADC2 采集 B 相电流。采样完成后，ADC1 的注入转换完成中断会调用 FOC_LOOP_CUR_Update()，进而调用 FOC_CURRENT_Update()，通过 HAL_ADCEx_InjectedGetValue() 读取两个 ADC 的转换结果。随后将 ADC 原始计数值减去零点偏置，并结合 ADC 参考电压、12 位分辨率、采样电阻阻值和运放放大倍

数，将其换算成实际电流。由于三相电机在没有中性线电流的情况下满足 $i_a + i_b + i_c = 0$，因此只需要采集 A、B 两相电流，就可以计算得到 C 相电流，不需要再使用第三路 ADC。最后将计算得到的三相电流保存到电流采样结构体中，提供给电流环进行 Clarke 和 Park 坐标变换，得到 d、q 轴反馈电流。整个电流采样模块的执行流程就是：上电初始化、零电流偏置校准、PWM 同步触发 ADC、读取采样结果、电流换算、第三相电流重构，最后将三相电流提供给 16kHz 电流闭环。

编码器角度测量和测速

在我的 ZqFOC 项目中，编码器角度测量和测速功能主要由 foc_encoder.c 文件实现，核心作用是通过增量式编码器获取转子的机械角度、电角度和实际转速，为 FOC 坐标变换以及速度、位置闭环提供反馈。我使用的是 MT6701 增量式 ABZ 编码器，分辨率为 1024PPR，通过 STM32G431 的 TIM3 配置为正交编码器模式，同时检测 A、B 两相信号的上升沿和下降沿，实现四倍频计数，因此电机每旋转一圈，定时器会产生 4096 个计数。这个文件主要包含以下几个函数。首先是 FOC_ENCODER_Init()，负责初始化编码器结构体，将 TIM3 句柄与编码器对象关联，清零角度和速度变量，并通过 HAL_TIM_Encoder_Start() 启动编码器计数。第二个是 FOC_ENCODER_UpdateAngle()，负责计算机械角度和电角度。程序首先读取 TIM3 的 CNT 计数寄存器，根据当前计数值除以每圈 4096 个计数，再乘以 2π，得到转子的单圈机械角度；接着根据电角度等于机械角度乘以电机极对数再加上电角度偏置的关系，计算实际电角度，并将结果归一化到 0 到 2π 范围内。当前工程配置的电机极对数为 7。第三个是 FOC_ENCODER_UpdateSpeed()，主要负责计算实际转速。我采用的是固定时间间隔内的编码器计数差分测速法，也就是 M 法测速。具体来说，通过 TIM6 每隔 1ms 触发一次速度环，在速度环中读取当前编码器计数值，减去上一次的计数值，得到这 1ms 内的计数增量，再根据编码器每圈 4096 个计数以及采样周期，将计数增量换算为 rpm。考虑到编码器计数器在 4095 和 0 之间循环，程序还对计数差值进行了回绕修正，避免计数器溢出造成转速计算错误。同时，由于低速时单位时间内的计数变化较少，差分测速容易产生量化波动，因此我增加了一阶低通滤波器，滤波时间常数设置为 5ms，通过当前测速值与上一时刻滤波结果的加权计算，得到更加平滑的速度反馈。第四个是 FOC_ENCODER_SetElectricalOffset()，用于设置编码器的电角度零点偏置。第五个是 FOC_ENCODER_CalibrateElectricalOffset()，用于根据单次对齐后的编码器读数计算电角度偏置；而当前工程实际使用的是第六个函数 FOC_ENCODER_CalibrateElectricalOffsetSamples()，它在启动对齐阶段采集 10 次机械角度，通过正弦和余弦求圆周平均，避免角度跨越零点时普通算术平均产生误差，再根据对齐时施加的固定电角度计算零点偏置。**这些函数在整个控制系统中相互配合：电机启动时先完成编码器电角度零点校准；正常运行后，16kHz 电流环调用角度更新函数，为 Park 和反 Park 变换提供电角度；1kHz 速度环调用测速函数，为速度 PI 控制器提供实际转速；位置环则利用机械角度作为位置反馈。**这样就实现了电机位置、转速与电流控制之间的闭环配合。

manager

在我的 ZqFOC 项目中，foc_manager.c 主要负责整个电机控制系统的初始化、启动、停止以及运行状态管理，相当于整个 FOC 控制系统的调度和管理模块。为了提高程序的模块化程度，我将编码器、电流采样、SVPWM、电流环、速度环和位置环分别封装成独立模块，然后通过 FOC_Motor_HandleTypeDef 结构体统一管理这些模块。这个文件主要包含以下几个函数。首先是 FOC_Motor_Init()，它在 main() 函数完成 STM32 外设初始化后调用，主要负责 ADC 校准、内部运放启动、编码器初始化、电流采样初始化、母线电压采样初始化、SVPWM 初始化以及三个 PI 控制环的初始化，同时在三相功率输出关闭的情况下完成零电流偏置校准，最后初始化 VOFA+ 串口通信。第二个是 FOC_Motor_Start()，主要负责电机启动。当收到启动指令后，程序首先检查电机是否处于停止状态、零电流校准是否成功以及母线电压是否有效，然后清除之前的控制目标和 PI 积分状态，启动三相 PWM 输出，并施加一个固定方向、幅值为 1V 的电压矢量，使转子逐渐对齐到指定电角度，随后将电机状态切换为 FOC_STATE_ALIGNING。第三个是 FOC_Motor_HandleAligning()，负责处理编码器对齐过程。程序首先保持固定磁场 1000ms，使转子稳定，然后采集 10 次编码器机械角度，通过圆周平均计算编码器电角度零点偏置。完成校准后撤去对齐电压，进入 FOC_STATE_ALIGN_RELEASE 状态。第四个是 FOC_Motor_HandleAlignRelease()，它会等待 800ms，让转子在撤去对齐电压后恢复稳定，然后重新同步编码器计数与测速历史、清零 PI 积分、重置控制目标，最后启动 ADC 注入转换中断和 TIM6 定时器中断，将电机状态切换为 FOC_STATE_RUNNING，正式进入闭环控制。如果当前是速度模式，还会设置初始目标转速为 100rpm。第五个是 FOC_Motor_ProcessAlignment()，它通过 switch 语句判断当前电机状态，分别调用对齐函数和对齐释放函数，实现非阻塞的状态机控制，不需要使用长时间的 HAL_Delay() 阻塞主循环。第六个是 FOC_Motor_Stop()，当收到停止指令时，首先将电机状态切换为停止，然后关闭 TIM1 的主输出使能 MOE，停止三相互补 PWM、ADC 注入采样和 TIM6 速度环中断，同时清零速度、电流目标及各个 PI 控制器的积分状态，防止再次启动时受到上一次控制结果的影响。第七个是 FOC_Motor_UpdateBusVoltage()，负责每隔 10ms 更新一次母线电压，供电流环电压限幅和 SVPWM 计算使用。最后是 FOC_Motor_Task()，它是整个管理模块的主循环任务，在 while(1) 中反复调用，依次执行母线电压更新、按键处理、状态机推进以及 VOFA+ 通信任务。这里需要强调的是，我采用了主循环与定时器中断相结合的架构：主循环主要负责状态管理、通信和非实时任务，而对实时性要求较高的控制算法通过硬件中断执行，其中 ADC 注入转换完成中断执行 16kHz 电流环，TIM6 定时器中断执行 1kHz 速度环，并每 5 次执行一次 200Hz 位置环。这样既保证了电流控制的实时性，也实现了电机从初始化、编码器对齐、闭环运行到停止的完整控制流程。

can通信

在我的 ZqFOC 项目中，CAN 通信主要用于实现 ESP32-S3 人机交互端与 STM32G431 电机控制端之间的双向数据交互，其中 ESP32-S3 负责通过 LVGL 界面发送电机控制指令并显示运行状态，STM32G431 负责接收指令、执行 FOC 控制算法并反馈电机的实际运行数据。整个系统采用经典 CAN 协议，通信波特率设置为 500kbit/s，使用 11 位标准帧 ID，每帧最多传输 8 字节数据。在 STM32 端，

通信功能主要由 foc_can.c 文件实现，首先通过 FOC_CAN_Init() 初始化 FDCAN1，配置接收过滤器，只接收 ID 为 0x100 的控制报文，并将符合条件的报文存入接收 FIFO0。随后在主循环中不断调用 FOC_CAN_ProcessRx()，通过 HAL_FDCAN_GetRxFifoFillLevel() 判断 FIFO 中是否存在待处理报文，再使用 HAL_FDCAN_GetRxMessage() 读取报文，检查报文 ID、帧类型以及数据长度，最后根据第一个字节的命令编号执行对应操作。例如 0x01 表示设置控制模式，0x02 表示设置目标转速，0x03 表示设置目标位置，0x04 表示启动电机，0x05 表示停止电机。对于速度指令，我将第二、第三个字节按照小端格式组合成一个 16 位有符号整数，表示目标转速，单位为 rpm，再通过 FOC_LOOP_SPD_SetSpeedRef() 更新速度环目标值；对于启动和停止指令，则分别调用 FOC_Motor_Start() 和 FOC_Motor_Stop()，复用现有的电机状态机，避免 CAN 通信模块直接操作底层 PWM 寄存器。另外，考虑到电机启动需要经过编码器对齐过程，程序还设计了目标指令缓存机制，确保启动对齐完成后再应用目标转速或位置。在数据反馈方面，STM32 通过 FOC_CAN_SendFeedback() 周期性发送四种报文，其中 0x180 表示电机运行状态和控制模式，0x181 表示实际位置和目标位置，0x182 表示 d、q 轴的实际电流和目标电流，0x183 表示实际转速和目标转速。程序每隔 25ms 发送一类反馈报文，因此每类数据的正常反馈周期为 100ms，并通过 HAL_FDCAN_AddMessageToTxFifoQ() 将报文提交到硬件发送 FIFO，发送缓冲区满时直接跳过，避免阻塞主循环。在 ESP32-S3 端，CAN 通信主要由 comm_can.c 文件实现，使用 ESP32 的 TWAI 控制器，并结合 FreeRTOS 消息队列完成数据接收与处理。 首先通过 comm_can_init() 初始化 TWAI 控制器、配置 500kbit/s 波特率、创建接收队列并注册发送和接收回调函数。当用户在 LVGL 界面上操作速度滑块或者启停按钮时，程序首先调用 comm_can_pack_command()，将命令编号和参数按照约定格式打包成 8 字节数据，再调用 comm_can_send_command()，通过 twai_node_transmit() 提交发送请求。发送完成后，can_tx_callback() 负责记录发送成功或失败的状态，并释放对应的发送缓冲区，其中 STOP 命令还使用了独立的发送缓冲区，避免被普通控制命令占用。当 STM32 返回运行数据时，ESP32 会进入 can_rx_callback() 接收回调，在中断上下文中调用 twai_node_receive_from_isr() 读取 CAN 报文，再通过 xQueueSendFromISR() 将报文存入 FreeRTOS 消息队列，而不在中断中直接进行界面刷新。随后 LVGL 任务通过 comm_can_receive() 从队列中取出报文，根据不同的报文 ID 调用 comm_can_parse_status()、comm_can_parse_speed()、comm_can_parse_current() 和 comm_can_parse_position() 等函数完成数据解析，最后更新界面上的转速、电流、位置和电机运行状态。整个 CAN 通信设计采用了控制与显示分离、中断接收与任务处理分离的方式，STM32 专注于实时电机控制，ESP32 专注于人机交互，同时通过报文过滤、数据校验、消息队列和非阻塞发送机制，提高了通信的可靠性，并尽量避免通信任务影响 16kHz 电流环和 1kHz 速度环的实时运行。

电流环和速度环

在我的 ZqFOC 项目中，电流环和速度环分别由 foc_loop_cur.c 和 foc_loop_spd.c 实现，二者采用串级控制结构，速度环负责根据转速误差计算目标转矩电流，电流环负责根据电流误差计算目标电压，最终通过 SVPWM 控制逆变器输出。 其中速度环运行频率为 1kHz，电流环运行频率为 16kHz，保证内环具有更快的响应能力。

首先是 foc_loop_spd.c 速度环模块，它主要包含三个函数。第一个是 FOC_LOOP_SPD_Init()，负责初始化速度环结构体，关联编码器对象和电流环对象，并初始化速度 PI 控制器，其中 PI 输出限幅设置为正负 2A，对应电机允许的最大 q 轴电流。第二个是 FOC_LOOP_SPD_SetSpeedRef()，负责设置目标转速，并将其限制在正负 2600rpm 范围内，正值表示正转，负值表示反转。第三个是 FOC_LOOP_SPD_Update()，这是速度环最核心的函数，由 TIM6 每隔 1ms 触发一次。程序首先调用 FOC_ENCODER_UpdateSpeed()，读取编码器计数变化并计算实际转速，经过一阶低通滤波后得到速度反馈值；然后将目标转速与实际转速相减，计算速度误差，并调用 FOC_PI_Update() 进行 PI 运算，得到 q 轴目标电流 i_q^*。由于 q 轴电流主要决定电磁转矩，因此速度 PI 的输出实际上就是通过调节目标转矩来控制电机加速或减速。最后调用 FOC_LOOP_CUR_SetReference()，将 d 轴目标电流设为零，将速度 PI 计算出的 q 轴目标电流传递给电流环。这里速度环并不直接控制 PWM，而是通过修改电流环的目标值实现间接控制。

接下来是 foc_loop_cur.c 电流环模块，它同样包含三个主要函数。第一个是 FOC_LOOP_CUR_Init()，负责初始化电流环结构体，将编码器、电流采样模块以及 SVPWM 模块的指针关联起来，同时初始化 d、q 两个独立的 PI 控制器，并设置各自的比例系数、积分系数和输出电压限幅。第二个是 FOC_LOOP_CUR_SetReference()，主要用于更新 d、q 轴目标电流，其中在当前正常速度控制模式下，$i_d^* = 0$，而 i_q^* 来自速度环 PI 的输出。第三个是 FOC_LOOP_CUR_Update()，这是整个 FOC 算法的核心函数，由 ADC1 注入转换完成中断以 16kHz 的频率调用。程序首先通过 FOC_CURRENT_Update() 读取 A、B 两相采样电流并重构 C 相电流，同时调用 FOC_ENCODER_UpdateAngle() 获取当前转子电角度。随后调用 FOC_MATH_Clarke() 将三相静止坐标系下的电流转换为 $\alpha\beta$ 两相静止坐标系电流，再调用 FOC_MATH_Park()，结合转子电角度将其转换为同步旋转坐标系下的 d、q 轴电流。接下来分别将 d、q 轴实际电流与目标电流比较，通过两个独立的 PI 控制器计算得到目标电压 u_d 和 u_q。由于逆变器实际能够输出的电压受到直流母线电压限制，因此程序会计算电压矢量幅值，并根据当前母线电压和电压利用系数进行限幅。如果电压矢量超过允许范围，就按照相同比例缩小 d、q 轴电压，保持电压矢量方向不变。完成限幅后，调用 FOC_MATH_InvPark()，将 d、q 轴电压转换为 $\alpha\beta$ 坐标系下的电压分量，最后调用 FOC_SVPWM_Update()，通过 Min-Max 法计算三相 PWM 占空比，并更新 TIM1 的 CCR1、CCR2、CCR3 比较寄存器，控制三相逆变桥的开关状态。

两个控制环的配合过程可以用设置目标转速为 1000rpm 来说明： 当 ESP32 通过 CAN 发送 1000rpm 的目标速度后，STM32 的速度环每隔 1ms 读取一次编码器反馈转速，计算速度误差，通过 PI 控制器产生目标 q 轴电流；与此同时，电流环每隔 62.5μs 执行一次，采集实际电流并计算 d、q 轴电流误差，再通过 PI 控制器产生目标电压，经过坐标变换和 SVPWM 更新三相 PWM 占空比，使电机产生相应的电磁转矩并逐渐加速。当实际转速接近 1000rpm 时，速度误差逐渐减小，速度 PI 会相应调节目标 q 轴电流，使电磁转矩与负载转矩达到平衡，从而稳定运行在目标转速附近。如果负载突然增加，电机转速出现下降，速度 PI 就会提高目标 q 轴电流，电流环快速跟踪这一目标，增大电磁转矩以抵消负载扰动。因此，速度环主要保证转速跟踪和抗负载扰动能力，电流环主要保证电磁转矩的快速、准确响应，采用外慢内快的串级结构，可以兼顾系统的动态响应和稳定性。

电角度对齐和补偿

在我的 ZqFOC 项目中，电角度对齐和补偿主要由 foc_manager.c 和 foc_encoder.c 两个文件配合实现，目的是确定编码器机械零点与电机转子磁极之间的相对位置，保证 FOC 控制所使用的电角度与实际转子磁场方向一致。 因为增量式编码器上电后只能通过计数得到相对于初始位置的机械角度，并不能直接知道转子永磁体的实际磁极方向，如果直接将机械角度乘以极对数作为电角度，就可能存在角度偏差，导致 Park 变换得到的 d、q 轴电流不准确，进而引起转矩下降、电流增大甚至电机抖动。因此，我在 foc_manager.c 中设计了编码器对齐状态机。电机收到启动命令后，首先调用 FOC_Motor_Start()，清零控制目标和 PI 积分状态，启动三相 PWM，并通过 FOC_SVPWM_Update() 在静止坐标系的 α 轴方向施加一个 1V 的固定电压矢量，使定子产生固定方向的磁场，转子在电磁转矩作用下逐渐转动并与这个磁场方向对齐。随后进入 FOC_STATE_ALIGNING 状态，保持固定磁场 1000ms，使转子充分稳定。对齐完成后，FOC_Motor_HandleAligning() 会调用编码器角度更新函数，间隔 2ms 采集一次机械角度，共采集 10 次，再调用 FOC_ENCODER_CalibrateElectricalOffsetSamples() 计算角度偏置。这里我没有直接采用普通算术平均，而是先对采集到的机械角度分别计算正弦和余弦，再通过 atan2() 求取圆周平均值，避免角度在 0° 和 360° 附近跨界时产生平均误差。得到平均机械角度以后，根据电角度等于机械角度乘以极对数再加上电角度偏置的关系，计算偏置量，公式为 $\theta_{\text{offset}} = \theta_{\text{align}} - p\theta_m$，其中当前电机极对数为 7，对齐目标电角度为 0。计算完成后，将偏置量保存到编码器结构体的 angle_e_offset 变量中，并归一化到 0 到 2π 范围。随后撤去对齐电压，进入 FOC_STATE_ALIGN_RELEASE 状态，等待 800ms，使转子恢复稳定，再重新同步编码器计数与测速历史，清零控制器积分状态，最后启动电流环和速度环，进入正常闭环运行。在正常运行过程中，16kHz 电流环每次调用 FOC_ENCODER_UpdateAngle() 时，都会根据当前编码器计数计算机械角度，再乘以极对数并加上之前校准得到的偏置量，从而获得补偿后的电角度，供 Park 和反 Park 变换使用。因此，我当前项目采用的是启动阶段的静态电角度零点校准，以及运行阶段的固定偏置补偿，并没有额外实现根据转速预测电角度的动态超前补偿。 通过这种方式，可以降低编码器安装位置和上电初始位置不确定性带来的角度误差，提高 FOC 电流解耦控制的准确性。

面经

1. static修饰全局变量和局部变量的区别
   在 C 语言中，static 可以修饰局部变量和全局变量。对于局部变量，使变量在整个程序运行期间一直存在，并且只初始化一次，每次进入函数时保留上次的值。对于全局变量，static 不改变生命周期，而是将其链接属性限制为内部链接，使其不能被其他源文件通过 extern 直接访问。
2. 进程和线程
   进程是操作系统进行资源分配和管理的基本单位，拥有独立的内存空间、代码和数据等资源；线程是 CPU 调度和执行的基本单位，同一进程中的多个线程可以共享进程的内存和资源，但各自拥有独立的栈和运行状态。一个进程可以包含多个线程
3.